

IA Aplicada com LLMs

Aula 02 — Escolhendo um modelo

Onde paramos

Na Aula 01, uma linha de código escolhia o modelo:

```
model="mistral-small-latest"
```

Ninguém questionou. O objetivo era ver texto na tela.

A partir de agora, **essa linha é uma decisão de arquitetura.**

O que essa linha decide

Ao mesmo tempo:

- a **qualidade** das respostas do seu sistema
- a **conta** que chega no fim do mês
- quanto o usuário **espera** pela primeira palavra
- quais **dados** saem da sua rede
- se o seu código ainda **funciona** em seis meses

A pergunta errada e a certa

✗ "*Qual é o melhor modelo?*" — não tem resposta. Só respostas erradas.

✓ A que tem resposta:

Qual modelo, com quais configurações, atende ao meu requisito de **qualidade nesta tarefa**, dentro do meu orçamento de **custo** e de **latência**, com a **política de dados** que eu preciso respeitar?

Roteiro

1. **O que diferencia um modelo** — parâmetros, MoE
2. **Quantização** — a conta, o nome, o que se perde
3. **Janela de contexto** — e o que acontece perto do limite
4. **Especialização** — instruct × raciocínio
5. **Licença e ciclo de vida** — os riscos que nenhum benchmark mostra
6. **Onde o modelo roda** — API, cloud, self-host, gateway

Objetivos

- **Decompor** o nome de um modelo e dizer o que cada parte implica
- **Calcular** o tamanho em memória dada a quantização
- **Explicar** o que a janela limita e o que degrada perto do limite
- **Distinguir** instruct × raciocínio e denso × MoE
- **Diferenciar** *open-weights* de *open-source*
- **Escolher** entre API, cloud, self-host e gateway

Parte 1

O que diferencia um modelo

Comece pelo nome

Nome	Tamanho	Variante	Quantização	Versão
mistral-small-latest	"small"	instruct	do provedor	alias móvel
ministral-3b-latest	3B	instruct	do provedor	alias móvel
magistral-small-latest	"small"	raciocínio	do provedor	alias móvel
llama3.1:8b-instruct-q4_K_M	8B	instruct	Q4	fixa

Duas leituras da tabela

1. Por API, a quantização não é sua. O provedor não conta com quantos bits serve — e pode mudar sem avisar.

2. Nome comercial não é unidade de medida.

O "small" de um laboratório pode ser maior que o "large" de outro.

O "small" deste ano é maior que o "medium" de dois anos atrás.

Não decore o catálogo: pergunte

```
resposta = client.models.list()

for modelo in sorted(resposta.data, key=lambda m: m.id):
    extras = modelo.model_extra or {}
    print(modelo.id,
          extras.get("max_context_length"),
          extras.get("capabilities"))
```

Responde três coisas: **quais** modelos a sua chave vê, **qual contexto**, e **quem suporta tool calling** (que a Aula 03 vai exigir).

Parâmetros: ordens de grandeza

Escala	Onde roda	Serve para
0,5B – 3B	notebook, celular	classificação, extração, roteamento
7B – 13B	1 GPU de consumo	uso geral, resumo, RAG simples
24B – 70B	GPU de datacenter, API	raciocínio, código, tarefas difíceis
100B+	API, cluster	o que os menores erram

Por que "maior" não é a resposta

- 1. Retornos decrescentes.** A perda cai como lei de potência: dobrar parâmetros não dobra qualidade.
- 2. Chinchilla (2022):** modelos estavam *grandes demais para os dados*. Um 8B moderno bate um 70B de três anos atrás.
- 3. A tarefa tem um teto.** Classificar em 5 rótulos: um 3B empata com um 123B. Você pagaria 40× pelo mesmo resultado.

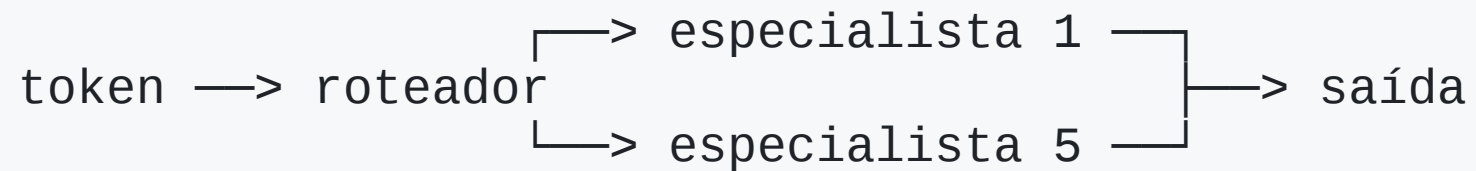
A pergunta certa

Qual é o menor modelo que ainda passa no meu teste?

E note o que ela pressupõe: que você **tem** um teste.

(É o benchmark próprio do deck 04.)

Denso × MoE



especialistas 2,3,4,6,7,8 → NÃO usados neste token

Mixtral 8x7B: ~47B de parâmetros **totais**, ~13B **ativos** por token.

Memória de modelo grande · velocidade de modelo médio.

A tag às vezes avisa: `qwen3.5:35b-a3b` = 35B totais, **3B ativos**.

Parte 2

Quantização

A conta

$$\text{memória} \approx \text{parâmetros} \times \frac{\text{bits}}{8}$$

Para um modelo de **7B**:

Precisão	Bits	Tamanho	Perda
float32	32	~28 GB	nenhuma
float16	16	~14 GB	desprezível
int8	8	~7 GB	pequena
Q4	4	~3,5 GB	moderada

Os modelos do Ollama vêm em **Q4 por padrão**.

O efeito que quase ninguém menciona

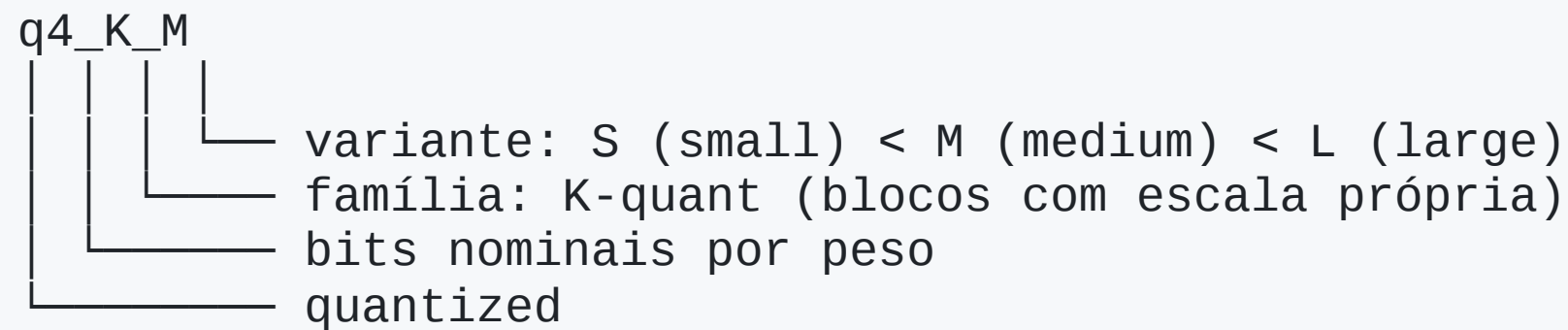
Quantizar não economiza só espaço — **acelera a geração.**

A razão não é aritmética, é **banda de memória**:

- para produzir *cada* token, o hardware **lê os pesos** da memória
- gerar é serial, um token por vez
- metade dos bytes por peso → metade do tráfego → quase o dobro da velocidade

Quantizar compra memória e velocidade, pagando com qualidade.

Decodificando q4_K_M



Tag	O que é
q8_0	8 bits, esquema legado : uma escala por bloco
bf16	sem quantização (no Ollama é bf16 , não fp16)
m1x	não é quantização: <i>build</i> para Apple Silicon

⚠ Nem todo tamanho tem todas as tags. **Consulte antes de baixar.**

Os bits nominais são um PISO

Três modelos reais, medidos:

Modelo	Params	Tag	Disco	Bits/param real	Nominal
Qwen3.6 (MoE)	36,0 B	q4_K_M	23 GB	5,1	4
Qwen3.5	873 M	q8_0	1,0 GB	9,2	8
Qwen2.5	494 M	q4_K_M	397 MB	6,4	4

Compare os dois q4_K_M : o 36B erra por **28%**, o 0,5B por **60%**.

Por que o pequeno erra mais

A matriz de **embeddings** fica em precisão alta.

Num modelo pequeno, ela é uma **fatia enorme** do total — num 0,5B com vocabulário grande, pode passar de um quarto dos parâmetros.

Num 36B, são migalhas.

Modelo pequeno é proporcionalmente mais incompressível.

O que se perde ao quantizar

Quantizar é **arredondar**, e o erro acumula ao longo das camadas.

O efeito raramente é "parou de funcionar". É pior:

- respostas curtas e fáceis continuam boas → **você não percebe nada**
- tarefas de várias etapas degradam primeiro
- a **cauda** piora: erra nos casos difíceis, que você não testou

Se quantizou, meça de novo.

Quando isso é decisão sua

Você	Quantização é
consome por API	problema do provedor — você não vê nem controla
hospeda o modelo	sua , e decide se cabe na GPU que você tem

Regra de bolso para self-host: some **20–30%** ao tamanho dos pesos, para o cache de KV e o *overhead* do runtime.

Parte 3

Janela de contexto

O que ela limita

O limite de tokens processados de uma vez — **entrada e saída somadas**.

Janela de 32k + prompt de 31.500 tokens = **500 tokens para responder**.

E o erro nem sempre é claro: pode vir resposta truncada

(`finish_reason="length"`) ou erro do provedor.

O custo escondido do contexto longo

A atenção compara cada token com todos os anteriores: custo **quadrático**.

Dobrar o contexto multiplica o trabalho por 4.

Duas coisas sobram para você:

- **você paga por token de entrada** — 100 mil tokens custam 100× mil, a cada chamada
- **o TTFT cresce** — o *prefill* processa o prompt inteiro antes do 1º token

"Janela de 1 milhão" é o que é **possível**, não o que é **recomendável**.

TTFT

TTFT (*time to first token*) = tempo até a **primeira palavra** aparecer.

```
|<----- TTFT ----->|<---- geração (tokens/s) ---->|  
| rede+fila | prefill      | t1 t2 t3 t4 ...           tN |
```

Merece nome próprio porque **não é** o tempo total: antes do 1º token, o modelo processa o prompt **inteiro** de uma vez.

Para o usuário, é o número que separa "respondeu" de "travou".

Lost in the middle

início do contexto
recuperação BOA



MEIO do contexto
recuperação DEGRADADA



fim do contexto
recuperação BOA



(*Liu et al., 2023*) — um viés em forma de U.

- instrução crítica **no começo ou no fim**, nunca soterrada
- **mais contexto não é melhor**
- se você precisa de 20 documentos, o problema é de **recuperação**

O histórico que some

O modelo não tem estado — quem mantém o histórico é o **seu programa**.

Quando a conversa passa da janela, algo é descartado. Muitos frameworks descartam o mais antigo **em silêncio**.

Resultado: *"o chatbot esqueceu o que eu falei no começo"* — **sem erro no log**.

Não é defeito do modelo. É uma decisão de engenharia que alguém tomou por você.

Parte 4

Especialização e tipos de modelo

Um modelo por trabalho

Especialização	Para que
Instruct / chat	o padrão: seguir instruções
Código	completar, explicar, corrigir
Raciocínio	problemas de várias etapas
Embeddings	texto → vetor (não gera texto)
Visão	imagem + texto (OCR, gráficos)
Moderação	classificar conteúdo, <i>em volta</i> do principal

Erro comum: achar que precisa do modelo **maior** quando a tarefa é **específica**.

Instruct × raciocínio

prompt → [instruct] —————→ resposta (~200 tokens)

prompt → [raciocínio] → pensamento → resposta (~200 tokens)
(~2.000 tokens, COBRADOS)

Test-time compute: em vez de treinar maior, gastar mais **na inferência**.

Duas tarefas, lado a lado

Instruct — *"Reescreva num tom formal: 'oi, o pedido tá atrasado, deu ruim com a transportadora'."*

→ o modelo **já sabe o que fazer**. É uma **transformação** da entrada, em uma passagem.

Raciocínio — *"Caixas de 12 canetas por R\$ 30, avulsas a R\$ 3,50. Preciso de 100 gastando o mínimo."*

→ não há transformação: há um **resultado a descobrir**.

A cadeia da segunda tarefa

1. caneta na caixa: $30/12 = 2,50$; avulsa: 3,50 → caixa é melhor
2. maximizar caixas: $8 \times 12 = 96$ canetas por **R\$ 240**
3. faltam 4 → **4 avulsas (R\$ 14 → 254)** ou **9ª caixa (+R\$ 30 → 270)**
4. $254 < 270$ → **8 caixas e 4 avulsas, R\$ 254**

O **passo 3 é uma armadilha**: a regra do passo 1 sugere a nona caixa.

Um instruct pressionado a responder direto acerta o raciocínio e **erra o fecho**.

O critério: etapas, não dificuldade

A resposta depende de um **resultado intermediário** que eu também preciso descobrir?

Tarefa	Parece	É
Soneto em decassílabos sobre logística	difícil	instruct
Traduzir contrato de 5 páginas	difícil	instruct
"Todo A é B, nenhum B é C. Algum A é C?"	fácil	raciocínio
Escolher entre 3 fretes com prazo e multa	fácil	raciocínio

O preço de pensar

- os tokens de raciocínio são **cobrados** como saída (a parte cara)
- a **latência** aumenta muito, e o streaming não salva
- em tarefa simples **não melhora nada** — só encarece
- se o `max_tokens` acaba no meio do raciocínio: **você paga tudo e não recebe resposta**

Raciocínio para decidir, instruct para redigir.

Parte 5

Licença e ciclo de vida

Open-weights ≠ open-source

Termo	Você recebe	Você não recebe
Open-source	pesos, código, dados, receita	—
Open-weights	os pesos	dados, receita, e às vezes o uso comercial
Proprietário	acesso por API	qualquer coisa que você hospede

A maioria dos "modelos abertos" é **open-weights**, e a licença **varia por modelo**: Apache 2.0, licença de pesquisa (proíbe uso comercial), cláusulas próprias.

Leia a licença **daquele** modelo, na versão de **hoje**.

O alias `-latest` é dependência sem versão

`mistral-small-latest` não é um modelo: é um **ponteiro** para o que o provedor considera atual.

Quando ele atualiza, o seu código chama **outra coisa** — sem você mudar uma linha.

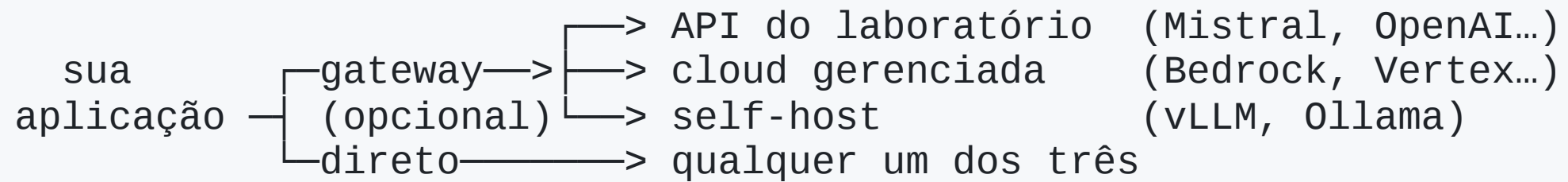
Ambiente	Recomendação
aula, protótipo	alias <code>-latest</code>
produção	versão datada fixa + testes de regressão

E some a **deprecação**: modelos são desligados. Precisa de plano de migração.

Parte 6

Onde o modelo roda

Quatro lugares



Como escolher

Modo	Custo	Privacidade	Operação	Quando
API	por token	dados vão ao provedor	~nenhuma	protótipo, volume baixo/médio
Cloud	por token, + caro	seu contrato/região	pouca	já tem compliance na nuvem
Self-host	GPU/hora, independe do volume	dados não saem	muita	volume alto e constante, dado sensível
Gateway	destino + margem	do destino	pouca	trocar provedor, roteamento, <i>failover</i>

O ponto de virada do self-host

A GPU roda 24h por dia, **com ou sem tráfego**. A API só cobra quando você chama.

- carga **esporádica** → API quase sempre mais barata
- carga alta e **constante** → self-host começa a compensar

Privacidade, mais que custo, é o argumento que justifica hospedar.

Faça a conta com o **seu** volume antes de decidir por instinto.

A interface virou padrão

```
# Mistral (o curso)
OpenAI(base_url="https://api.mistral.ai/v1", api_key=...)
modelo = "mistral-small-latest"

# Ollama local – mesmo código, outro endereço
OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
modelo = "qwen3.6:35b"
```

chat/completions é o denominador comum do mercado.

Portabilidade é decisão de arquitetura

Isole endpoint e modelo em um lugar → trocar de provedor é mudar 2 linhas.

Espalhe `model="mistral-small-latest"` **por 30 arquivos** → você se amarrou sem precisar.

No laboratório desta aula, os scripts `00` – `05` leem do `.env`:

```
cp .env.ollama .env      # trocou de provedor; nenhuma linha de código muda
```

 **Compatível não é idêntico** — o comportamento não é portátil (deck 02).

Nem tudo é configuração

Nos scripts 06 e 07, os modelos ficam **fixos no código**. De propósito.

Configuração é o que muda entre ambientes sem mudar o que o programa faz.

Ali o modelo é o **objeto do experimento** — o 06 compara pequeno, médio e grande com o **preço de cada um na linha ao lado**.

Uma tabela de custo em que ninguém sabe quem entrou não vale nada.

429 : o erro da aula prática

```
for tentativa in range(5):  
    try:  
        return client.chat.completions.create(**kwargs)  
    except RateLimitError:  
        time.sleep(2 ** tentativa)    # 1, 2, 4, 8, 16 s
```

Backoff exponencial. A turma inteira bate na mesma API.

Vira requisito obrigatório quando um agente faz dezenas de chamadas por tarefa.

Síntese (1/2)

- A linha `model="..."` tem 4 eixos: **qualidade, custo, latência, dados**
- **Não decore o catálogo** — `client.models.list()`
- Mais parâmetros → **retornos decrescentes**. Pergunte pelo **menor** que passa
- **MoE**: memória de grande, velocidade de médio
- **Quantização** $\approx$ params $\times$ bits/8 — e **acelera** (banda de memória)
- `q4_K_M` = 4 bits nominais, K-quant, variante média. Nominal é **piso**: de 28% a 60% acima, pior quanto **menor** o modelo
- A perda aparece **na cauda**. Se quantizou, **meça de novo**

Síntese (2/2)

- A **janela** limita entrada + saída, custa quadrático, e degrada no **meio**
- Chat longo estoura a janela e o histórico é descartado **em silêncio**
- Existe **um modelo por trabalho** — o especializado pequeno ganha do genérico grande na tarefa dele
- **Raciocínio para decidir, instruct para redigir**
- **Open-weights ≠ open-source**; `-latest` é dependência sem versão
- Self-host compensa com volume **constante**; privacidade justifica mais que custo
- `chat/completions` é padrão → portabilidade sai de graça, se você isolar
- **429** é **esperado**: backoff desde o primeiro dia